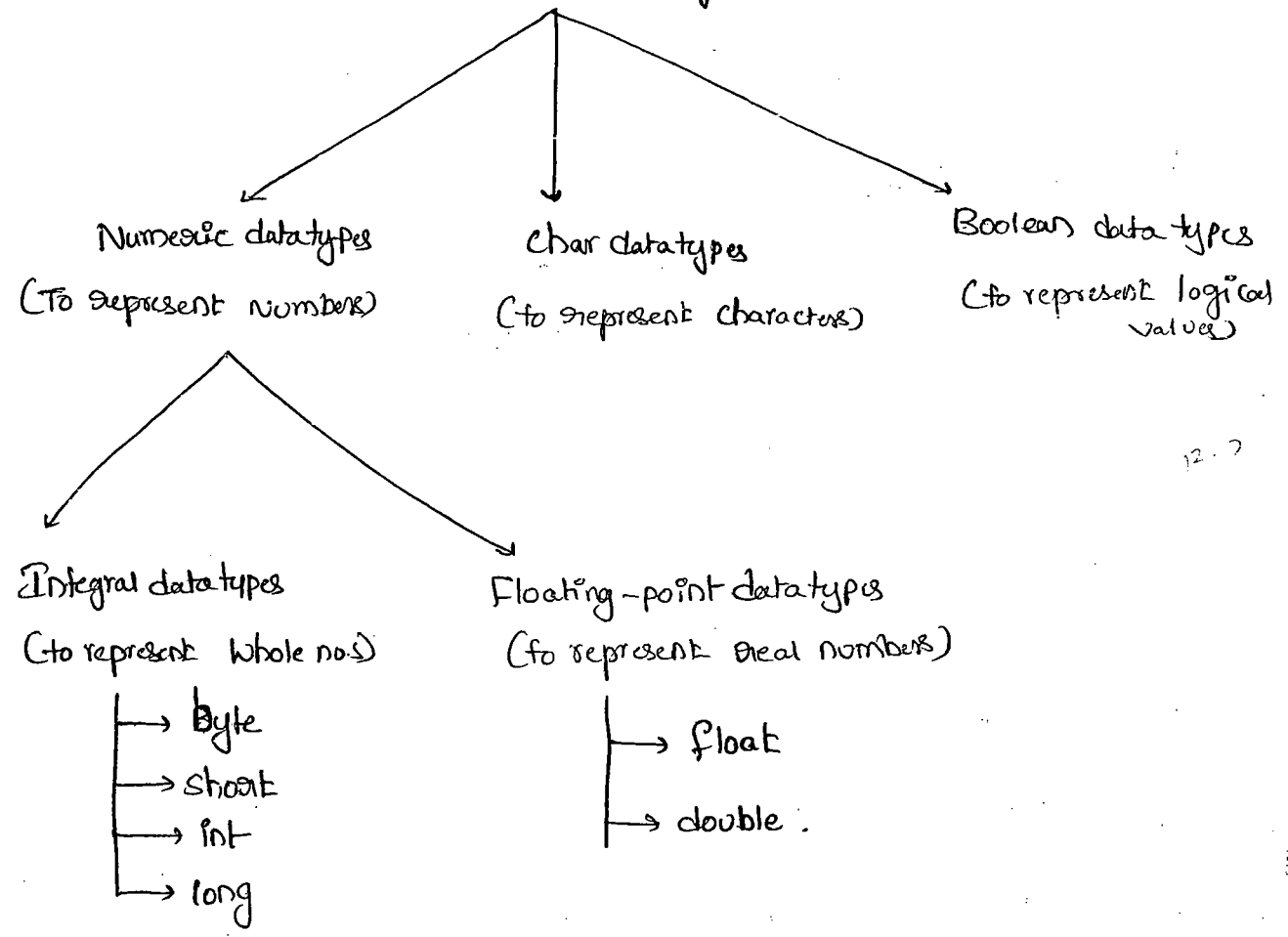
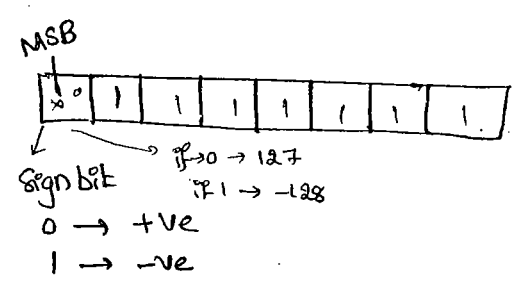


Primitive data types (8)



① Byte :-

Size = 8-bits (or 1 Byte)
 Max-value = 127
 Min-value = -128
 Range = -128 to +127



- The most Significant Bit is called "Sign bit". 0 means +ve value, 1 means -ve value.
- +ve numbers represented directly in the memory where as -ve numbers represented in 2's Complement form.

Ex:-
 ✓ byte b = 100;
 ✓ byte b = 127;

X byte b = 130; C.E! - possible loss of precision

Found: int

Required: byte

X byte b = 123.456; C.E! - PLP

Found: double

Required: byte

X byte b = true; C.E! - ~~PLP~~ incompatible types

Found: boolean

Required: byte

X byte b = "durga"; C.E! - incompatible types

Found: ~~String~~ .lang.String

Required: byte.

→ byte datatype is best suitable if we want to handle data in terms
of Streams either from the file or from the Network.

② Short :-

Size : 2-bytes (16-bits)

Range : -2^{15} to $2^{15}-1$

$[-32768 \text{ to } 32767]$

Eg! ✓ Short s = 32767

✓ Short s = -32768

X Short s = 32768 C.E! - PLP
Found: int
Required: short

X Short S = 123.456 C.E:- PLP

Found: double

Required: Short

X Short S = true C.E:- Incompatible types

Found: boolean

Required: Short

→ Most frequently used datatype in Java is Short

→ Short datatype is best suitable if we are using 16-bit processors like 8086 but these processors are completely outdated & hence corresponding Short datatype is also outdated.

③ int :-

→ The most commonly used datatype is int

Size: 4-bytes

Range: -2^{31} to $2^{31}-1$

$[-2147483648 \text{ to } 2147483647]$

Note:-

→ In C language the size of int is varied from platform to platform for 16-bit processors it is 2-bytes but for 32-bit processors it is 4-bytes

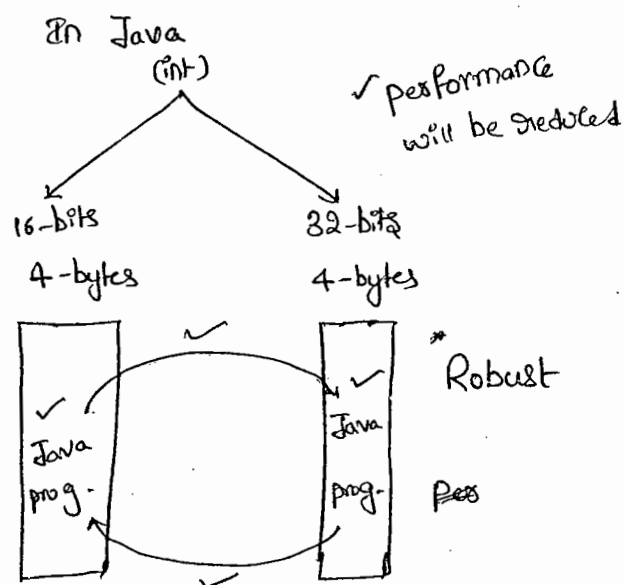
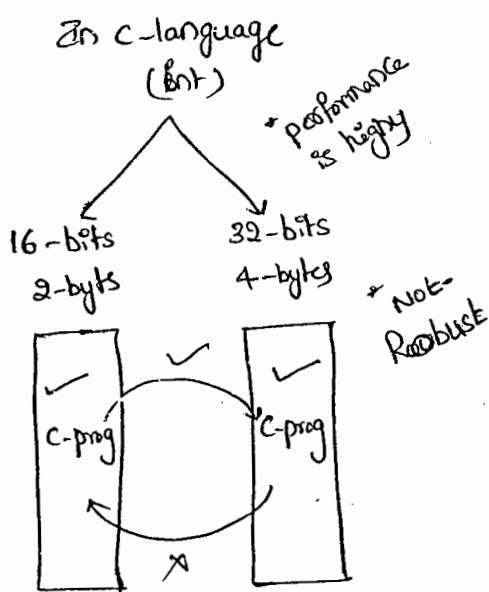
* The main advantage of this approach is read & write operation ^{we can} perform very efficiently and performance will be improved. But the main

* disadvantage of this approach is the chance of ~~failure~~ ^{failing} C program

is very very high if we are changing platform. Hence C-language is not considered as Robust.

→ But in Java the size of int is always 4-bytes irrespective of any platform. * The main advantage of this approach is the chance of failing Java program is very very less, if we are changing underlying platform. Hence Java is considered as Robust language.

* But the main disadvantage in this approach is read & write operations will become costly & performance will be reduced.



13/02/11

4) long :-

→ When ever int is not enough to hold big values then we should go for long data type.

Ex (1):- To represent the amount of distance travelled by light in 1000 days int is not enough Compulsary we should go for long type

Ex (2):- long $l = 1,23,000 \times 60 \times 60 \times 24 \times 1000$ miles;

2)

Ex(2):-

To Count the no. of characters present in a big file. int may not enough Compulsary we should go for long data type.

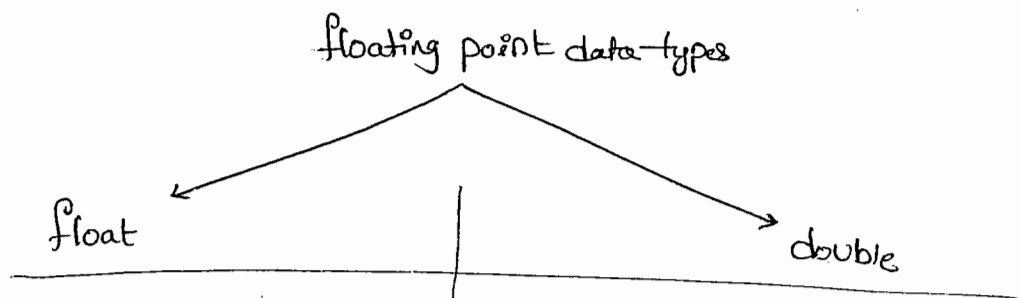
Size = 8 bytes

Range = -2^{63} to $2^{63}-1$

Note:-

- All the above data-types (byte, short, int, long) ment for representing whole values.
- If we want to represent real numbers Compulsary we should go for floating point data types.

Floating Point data types :-



- 1) Size : 4-bytes
- 2) Range : $-3.4e38$ to $3.4e38$

3) If we want 5 to 6 decimal places of accuracy then we should go for float

4) float follows single precision

- 1) Size : 8-bytes

2) Range : $-1.7e308$ to $1.7e308$

3) If we want 14 to 15 decimal places of accuracy then we should go for double.

4) double follows double precision

Boolean data type :-

Size : Not Applicable (Virtual machine dependent)

Range : Not Applicable [But allowed values are true/false]

Q) Which of the following boolean declarations are valid

X 1) boolean b = 0; C.E:- incompatible types

~~found~~ : int
required : boolean

✓ 2) boolean b = true;

X 3) boolean b = True; C.E:- Can't find symbol

Symbol : Variable True

Location : class Test

X 4) boolean b = "false" C.E:- incompatible types

~~found~~ : java.lang.String

required : boolean

✓ 5) boolean True = true
boolean b = True
System.out.println(b); true

C++

int x = 0;

if(x)

{
System.out.println("Hello");

else

{
System.out.println("Hi");

}

in Java
X

in Java
X

while(1)

{
System.out.println("Hello");

}

in C++
✓

C.E:- incompatible types

~~found~~ : int

required : boolean

→ The only allowed values for the boolean datatypes are 'true' or 'false' where false is important.

Char datatype :-

→ In ~~low~~ languages like C & C++ we can use only ASCII characters and to represent all ASCII characters 8-bits are enough. hence char size is 1-byte.

→ But in java we can use unicode characters which covers world wide all alphabets sets. The no. of unicode characters is > 256 & hence 1-byte is not enough to represent all characters compulsory we should go for 2-bytes.

Size : 2-bytes

Range : 0 to 65535

Summary of primitive data types :-

datatype	Size	Range	Corresponding Wrapper classes	default value
byte	1-byte	-2^7 to 2^7-1 [-128 to 127]	Byte	0
Short	2-bytes	-2^{15} to $2^{15}-1$ [-32768 to 32767]	Short	0
int	4-bytes	-2^{31} to $2^{31}-1$ [-2147483648 to 2147483647]	Integer	0
long	8-bytes	-2^{63} to $2^{63}-1$	Long	0
float	4-bytes	$-3.4e38$ to $3.4e38$	Float	0.0
double	8-bytes	$-1.7e308$ to $1.7e308$	Double	0.0
Char	2-bytes	0 to 65535	Character	0 [represents blank space]
boolean	NA	NA [true/false are allowed]	Boolean	false (True in C++)

Literals :-

→ A Constant value which can be assigned to the variable is called "Literal".

Ex:- `int x = 10;`

datatype/keyword Name of variable/identifier Constant value/Literal.

Integral Literals :-

→ For the Integral data types (byte, short, int, long) the following are various ways to specify Literal value

1) decimal literals:-

allowed digits are 0 to 9

Ex:- `int x = 10;`

2) Octal literals:-

→ allowed digits are 0 to 7

→ literal value should be prefixed with "0" [zero]

Ex:- `int x = 010;`

3) Hexadecimal literals:-

→ allowed digits are 0 to 9, a to f or A to F

→ For the Extra digits we can use both upper case & lower case.

This is one of very few places where Java is not case sensitive.

→ Literal value should be prefixed with 0x or 0X

8

Ex^o- int x = 0x10

(1)

int x = 0x10

→ These are the only possible ways to specify integral literal.

Ex^o- class Test

{

 p.s.v.m (String args)

{

 int x = 10;

 int y = 010;

 int z = 0x10;

 S.o.pln(x + "-----" + y + "-----" + z);

 10

 8

 16

 };

$$(10)_8 = (?)_{10}$$

$$0 \times 8^1 + 1 \times 8^0 = 8$$

$$(10)_{16} = (?)_{10}$$

$$0 \times 16^1 + 1 \times 16^0 = 16$$

Ques 111

Q) which of the following declarations are valid.

✓ ① int x = 10;

✓ ② int x = 066;

X ③ int x = 0786; C.E: integer number too large

✓ ④ int x = 0XFACE; 64206

X ⑤ int x = 0XBEEF; C.E- (after Bc) : Excepted

✓ ⑥ int x = 0XBca; 3050

→ By default Every integral literal is of int type but we can Specify Explicitly as long type by Suffixing with l or L.

Ex:-

✓ 1) `int i = 10;`

X 2) `int i = 10l;` C.E! PLP
found = long
Required = int

✓ 3) `long l = 10l;`

✓ 4) `long l = 10;`

→ There is no way to Specify integral literal is of byte & short types Explicitly.

→ If we are assigning integral literal to the byte variable & that integral literal is within the range of byte then it treats as byte literal automatically. Similarly short literal also.

Ex! - `byte b = 10;` ✓

`byte b = 130;` X C.E! PLP
found = int
Required = byte

Floating point Literals:-

→ Every floating point literal is by default double type & hence we can't assign directly to float variable.

→ But we can Specify Explicitly floating point literal is the float type by Suffixing with 'f' or 'F'.

Ex! X `float f = 123.456;` P.L.P
found = double
Required = float
✓ `float f = 123.456f;`
✓ `double d = 123.456;`

→ We Can Specify floating point literal Explicitly as double type & by Suffixing with d or D.

Ex: ✓ double d = 123.4567D;

X float f = 123.4567d; C.E:- PLP

found: double

required: float

→ We Can Specify floating point literal only in decimal form & we Can't Specify in Octal & Hexa decimal form.

Ex:-

✓ 1) double d = 123.456;

✓ 2) double d = 0123.456; o/p:- 123.456

X 3) double d = 0x123.456; C.E:- malformed floating point literal

Q) which of the following floating point declarations are valid?

X 1) float f = 123.456;

✓ 2) double d = 0123.456;

X 3) double d = 0x123.456;

✓ 4) double d = 0xface; // 64206.0

✓ 5) float f = 0xBea;

✓ 6) float f = 0642; // 418.0

Because these 3 are not floating point
So, that values are taking int type.

→ We Can assign integer literal directly to the floating point datatypes & that integer literal can be specified either in decimal form or Octal form or hexa decimal form.